

12

INTERACTIVE AUTHENTICATION



When you authenticate to your Windows system, you'll usually access a login interface, enter your credentials, and be greeted with the desktop. But quite a lot happens behind the scenes to make this authentication process work. *Interactive authentication* is the mechanism that converts a set of credentials into a `Token` object that you can use to interact with authorization systems, such as access checks.

Windows uses many types of interactive authentication for a variety of purposes. For example, it uses one type when a user creates an interactive desktop and another when the user has provided credentials to a network-facing service. We'll begin this chapter by exploring how Windows creates your interactive desktop when you authenticate to a Windows system. We'll then cover how this interactive authentication is implemented through the `LsaLogonUser` API. Finally, we'll look at the various types of interactive authentication, the differences between them, and when they might be used.

Creating a User's Desktop

The most common way of interacting with a Windows system is via the user interface on a desktop. **Figure 12-1** summarizes the process of creating a user's desktop.

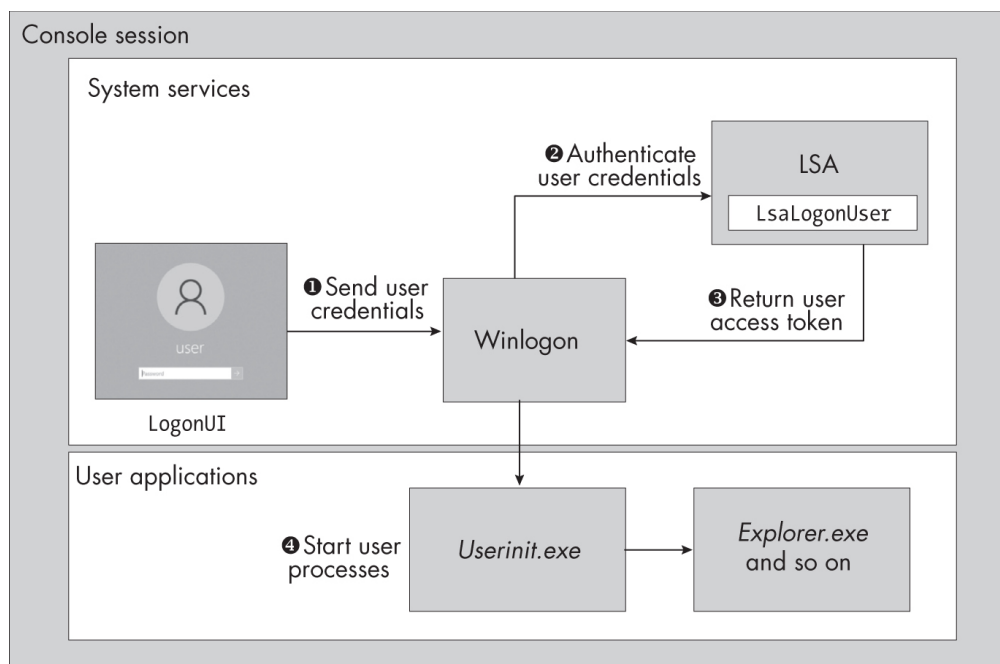


Figure 12-1: An overview of interactive desktop creation

When the Windows system starts, the session manager creates a console session, as described in [Chapter 3](#). In this console session it starts an instance of the Winlogon process, which gathers credentials and starts the new user's processes once they're authenticated. The Winlogon process then creates the LogonUI process to display a UI. The LogonUI process reads the credentials from the user and passes them back to Winlogon ①.

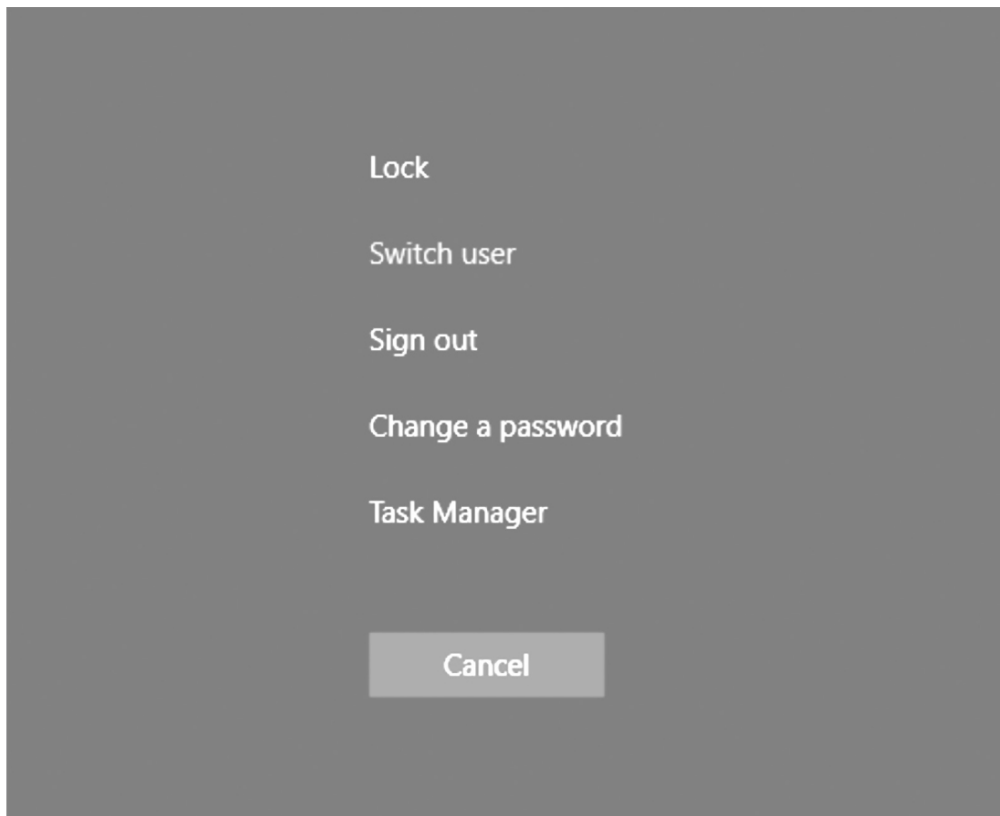
Next, the Winlogon process sends the credentials to the LSA's `LsaLogonUser` API to verify that they're correct ②. If the user has successfully authenticated, a token representing the user's identity is returned to Winlogon ③. The console session can then be reconfigured for the user, a process that includes creating a window station and desktop and spawning the user initialization process using the user's token ④.

The `LsaLogonUser` API directly supports the most common type of credential, a username and password pair. However, Windows allows many other local authentication factors as well, such as biometric data (for example, a scanning of the user's face) or a simple PIN. To handle these, Winlogon loads a credential provider when needed. Each provider is responsible for mapping its credential type to one that `LsaLogonUser` supports to get the token.

THE SECURE ATTENTION SEQUENCE

One of the original security features in Windows NT was the *secure attention sequence (SAS)*, which a user could invoke by pressing CTRL-ALT-DELETE. The handling of this key chord was built into the operating system, so applications couldn't block it. When the key chord was pressed, the system would notify Winlogon, which would switch the desktop to display an authentication prompt or an options menu. By making it impossible to block the SAS, Windows ensured it was safe for a user to enter their credentials into the computer.

The latest versions of Windows don't use the SAS as the LogonUI by default, although it's still a configurable option. If you're already authenticated and you press CTRL-ALT-DELETE, the system will switch to the Winlogon desktop to display the menu shown here.



The LsaLogonUser API

We know the basics of how to create a desktop on Windows. Now let's dig into how the `LsaLogonUser` API implements the interactive authentication service for Winlogon and other applications on the local system. This API might seem quite complex, but it really requires only three pieces of information from an application to authenticate a user:

- The logon type requested
- The security package identifier
- The user's credentials

The API uses the *logon type* to accommodate different authentication scenarios. **Table 12-1** lists the logon types most commonly used by applications.

Table 12-1: Common Logon Types

Logon type	Description
Interactive	Interact with a local desktop.
Batch	Run as a background process, even if no desktop is available.
Service	Run as a system service.
Network	Interact with the system from a network client.
NetworkCleartext	Perform network authentication, but store the user's credentials for later use.
NewCredentials	Clone the caller's token and change network user credentials.
RemoteInteractive	Interact with a desktop via the Remote Desktop Protocol.
Unlock	Verify the user's credentials for unlocking the desktop.

`Unlock` is a special type that Winlogon uses to verify a user's credentials on the lock screen, and it isn't typically used by applications directly. We'll come back to some of the other logon types later in the chapter.

Windows abstracts the details of authentication to a *security package*, which provides a standardized interface to an authentication protocol. The authentication protocol is a formal process that takes a set of creden-

tials and verifies that they're valid. It also provides a mechanism to return information about the verified user, such as their group memberships. We also sometimes refer to a security package as a *security support provider (SSP)*.

We can enumerate the available security packages using the `Get-LsaPackage` PowerShell command, as shown in [Listing 12-1](#).

```
PS> Get-LsaPackage | Select-Object Name, Comment
Name                                     Comment
----                                     -
Negotiate                               Microsoft Package Negotiator ❶
NegoExtender                            NegoExtender Security Package
Kerberos                                Microsoft Kerberos V1.0
NTLM                                     NTLM Security Package ❷
TSSSP                                   TS Service Security Package
pku2u                                    PKU2U Security Package
CloudAP                                 Cloud AP Security Package
WDigest                                 Digest Authentication for Windows
Schannel                                Schannel Security Package
Microsoft Unified Security Protocol Provider Schannel Security Package
Default TLS SSP                         Schannel Security Package
CREDSSP                                 Microsoft CredSSP Security Provider
```

Listing 12-1: Enumerating the supported security packages

Applications typically access a security package via a more generic API that is agnostic to the authentication protocol used. For example, `LsaLogonUser` works across multiple different packages by accepting a unique identifier for the package to use. A security package can also implement a network authentication protocol, which we'll cover in more depth in the following chapters.

The most widely used security packages for local authentication are *Negotiate* ❶ and *NT LAN Manager (NTLM)* ❷. The NTLM authentication protocol was introduced in Windows NT 3.1, and it's also sometimes referred to as the *Microsoft Authentication Package V1.0* in documentation. The Negotiate package can automatically select between different authentication protocols, depending on the circumstances. For example, it might select NTLM if authenticating locally to the SAM database or Kerberos when authenticating to a domain.

The supported credential types depend on the security package being used for the authentication. For example, NTLM supports only username and password credentials, whereas Kerberos supports X.509 certificates and smart card authentication in addition to a username and password.

Local Authentication

Let's explore how the `LsaLogonUser` API authenticates a user in more detail.

Figure 12-2 gives an overview of this process for a user in the local SAM database.

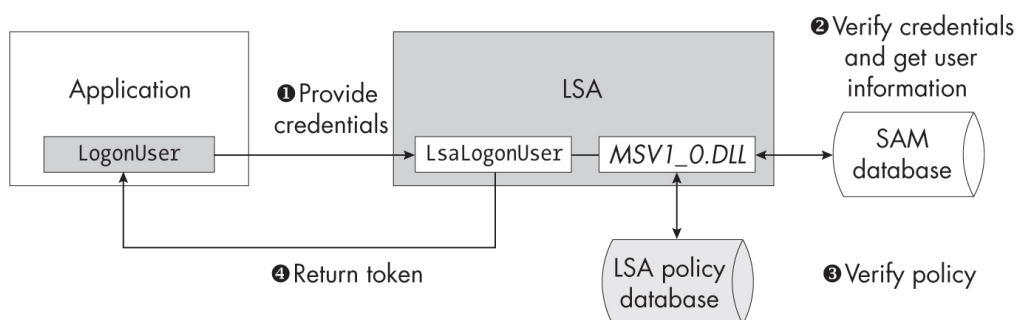


Figure 12-2: The local authentication process using `LsaLogonUser`

Due to the complexities of the `LsaLogonUser` API, it's more common for an application to use a simpler API provided by the system. For example, the `LogonUser` API accepts a username, a domain name, a password, and the logon type and formats the parameters appropriately for the underlying security package.

It then forwards these parameters, including the user's credentials, to the `LsaLogonUser` API in the LSA process **1**. The API in turn forwards the authentication request to the chosen security package, which in this case is the NTLM package implemented in the `MSV1_0.DLL` library.

The security package checks whether the user exists in the local SAM database. If it does, the user's password is converted to an NT hash (discussed in **Chapter 10**), and then it is compared against the value stored in the database **2**. If the hashes match and the user account is enabled, the authentication proceeds and the user's details, such as group membership, are read from the SAM database for the authentication process to use.

Now that the security package knows the user's group membership and account details, it can check whether the local security policy allows the user to authenticate ③. The main policy checks whether the logon type requested is granted an account right. **Table 12-2** lists the logon types and the account rights the user must be granted in order to authenticate. Note that the `NewCredentials` logon type doesn't need a specific account right; we'll cover why in the "Network Credentials" box on page 407.

Table 12-2: Logon Types and Associated Allow and Deny Account Rights

Logon type	Allow account right	Deny account right
Interactive	<code>SeInteractiveLogonRight</code>	<code>SeDenyInteractiveLogonRight</code>
Batch	<code>SeBatchLogonRight</code>	<code>SeDenyBatchLogonRight</code>
Service	<code>SeServiceLogonRight</code>	<code>SeDenyServiceLogonRight</code>
Network	<code>SeNetworkLogonRight</code>	<code>SeDenyNetworkLogonRight</code>
NetworkCleartext	<code>SeNetworkLogonRight</code>	<code>SeDenyNetworkLogonRight</code>
NewCredentials	N/A	N/A
RemoteInteractive	<code>SeRemoteInteractiveLogonRight</code>	<code>SeDenyRemoteInteractiveLogonRight</code>
Unlock	The same as Interactive or <code>RemoteInteractive</code>	The same as Interactive or <code>RemoteInteractive</code>

If the user doesn't have the necessary account right granted or is explicitly denied the right, the authentication will fail. There can be other limitations on authentication, as well; for example, you could configure a user so that they're allowed to authenticate only between certain times, or even only on certain days of the week. If the user doesn't meet one of the policy requirements, the security package will reject the authentication.

If the user's credentials are valid and the policy permits them to authenticate, the LSA can create a token using the `NtCreateToken` system call based

on the information about the user and their privileges extracted from the SAM and LSA policy databases ④. The application receives a handle to a token, which the user can subsequently use for impersonation or to create a new process within the limits of the assignment, as described in [Chapter 4](#).

Domain Authentication

Authenticating a user to a domain controller is not significantly different from local authentication, but it's still worth highlighting the small distinctions. [Figure 12-3](#) shows the domain authentication process.

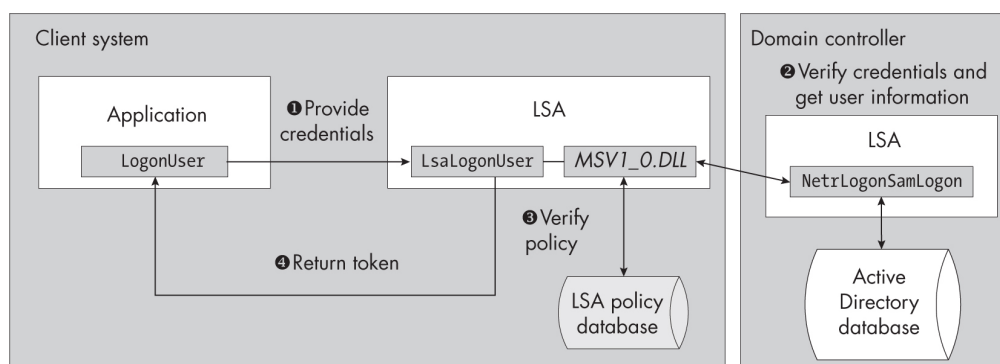


Figure 12-3: The domain authentication process using LsaLogonUser

The domain authentication process starts in the same manner as local authentication. The application provides the credentials and other parameters to the LsaLogonUser API running in the LSA process ①. At this point, it's likely that API will use the Negotiate security package to select the most appropriate security package to authenticate with.

In this example, it once again uses the NTLM security package, which is easy to understand. However, in a modern Windows network, you're more likely to find Kerberos used. Interactive authentication with Kerberos is much more complex, so I'll wait until [Chapter 14](#) to provide details about it.

Windows also supports online authentication protocols, such as those for Microsoft and Azure Active Directory accounts. Authentication for these accounts uses the CloudAP security package, which Negotiate will select automatically if it's the best security package to use. Details of this selec-

tion process are beyond the scope of this book, although we'll cover some aspects of Negotiate in [Chapter 15](#).

The NTLM security package once again generates the NT hash, but instead of consulting the local SAM database, it determines the domain controller for the user's domain. It then forwards the authentication request containing the user's name and NT hash to the domain controller's NetrLogonSamLogon API using the *Netlogon* network protocol.

While Windows has deprecated the Netlogon protocol for primary domain authentication, it has not removed the protocol in the latest versions. Not removing legacy features can result in important security issues as technology becomes obsolete and security expectations change. For example, CVE-2020-1472, dubbed *ZeroLogon*, was a serious vulnerability in the Netlogon protocol that allowed unauthenticated users to compromise the entire domain network due to a flaw in the weak cryptography used by the protocol.

The domain controller verifies the user's credentials in the domain's user database ❷. For modern versions of Windows, this is Active Directory, not a SAM database. The user must also be enabled for the authentication to succeed. If the hashes match, the user's information is extracted from Active Directory and returned to the client system.

Once the user's credentials have been validated, the client system can verify its local policy ❸ to determine whether the user is permitted to authenticate based on the logon type and other restrictions, such as time limits. If every check succeeds, the LSA generates the token and returns it to the application ❹.

CACHED DOMAIN CREDENTIALS

What happens to a Windows system that is connected to a domain when the enterprise network is disconnected or otherwise unavailable? If authentication relies on being able to contact a domain controller over the network, how could you authenticate to the system to change the network configuration? You could ensure that every user had a separate local user account to deal with this issue, but that isn't a very satisfactory option.

To solve this problem, the LSA stores a cache of recently used domain credentials. Each time a successful domain authentication occurs, the LSA caches the credentials. The next time the user authenticates to the system, if the domain authentication fails because the domain controller is no longer accessible, the LSA can check whether the credentials used match any of the values stored in the cache. If it finds a match, it will grant access to the system. However, the LSA will also keep trying to contact the domain controller to verify the user's credentials. This is especially important for Kerberos, because without contact with the domain controller, the user won't be able to access any network resources.

I mentioned in the previous chapter that these cached credentials are stored in the *SECURITY* registry hive. We won't delve into the details of this storage, as it could easily change between versions of Windows.

Logon and Console Sessions

Once the `LsaLogonUser` API has verified the user's credentials, it can create an initial token for the user. Before it can make a call to `NtCreateToken`, however, the LSA must set up an associated logon session. We discussed the logon session in [Chapter 4](#), in the context of the token's authentication ID, but it's worth going into more depth about what it contains.

Let's begin by querying the LSA for all current logon sessions using the `Get-NtLogonSession` PowerShell command, as shown in [Listing 12-2](#). You should run this command as an administrator to display all the logon sessions on the system.

```
PS> Get-NtLogonSession | Sort-Object LogonId
```

LogonId	UserName	LogonType	SessionId
-----	-----	-----	-----
❶ 00000000-000003E4	NT AUTHORITY\NETWORK SERVICE	Service	0
00000000-000003E5	NT AUTHORITY\LOCAL SERVICE	Service	0
❷ 00000000-000003E7	NT AUTHORITY\SYSTEM	UndefinedLogonType	0
❸ 00000000-00006A39	Font Driver Host\UMFD-0	Interactive	0
00000000-00006A96	Font Driver Host\UMFD-1	Interactive	1
00000000-0000C5E9	Window Manager\DWM-1	Interactive	1
❹ 00000000-00042A51	GRAPHITE\user	Interactive	1
00000000-00042AB7	GRAPHITE\user	Interactive	1

00000000-000E7A72	Font Driver Host\UMFD-3	Interactive	2
00000000-000E7CF2	Window Manager\DWM-3	Interactive	2

Listing 12-2: Displaying all current logon sessions

We can see that the first two sessions are for service accounts ❶, as indicated by the `LogonType` value. Oddly, the third session is also a service account, for the `SYSTEM` user, but notice that the `LogonType` is undefined ❷. This is because the kernel creates the `SYSTEM` logon session before the LSA process is started, which means no authentication has taken place.

The rest of the logon sessions are for interactive accounts, as indicated by the `Interactive` logon type ❸. Only one user is authenticated ❹; the other accounts belong to system processes such as the user-mode font driver (UMFD) and the desktop window manager (DWM). We won't cover these system processes in any detail. Observe that the current user has two logon sessions. This is because of UAC, introduced in [Chapter 4](#); we'll come back to why UAC generates two sessions in "Token Creation" on page 407.

Notice also that a `SessionId` is shown for each logon session, in addition to the authentication identifier (`LogonId`) that identifies the account. This is the console session ID. It's important not to confuse the logon session and console session types. As this output shows, it's possible for a single console session to host multiple separate logon sessions, and for a single logon session to be used across multiple console sessions.

The LSA stores the console session ID originally associated with the logon session when it was created. In [Listing 12-3](#), we query the LSA for all current console sessions with `Get-NtConsoleSession`. This behavior allows multiple users to share the same console and desktop.

```
PS> Get-NtConsoleSession
SessionId  UserName          SessionName        State
-----
0          GRAPHITE\user     Services           Disconnected
1          GRAPHITE\user     31C5CE94259D4006A9E4#0 Active
2          Console          Console            Connected
```

Listing 12-3: Displaying all current console sessions

The `SessionName` column indicates where the console session is connected. Session 0 is a `Services` console, meaning it's used only for system services. The `State` column indicates the state of the UI. For session 0 this is set to `Disconnected`, as there is no UI displayed.

Session 1 is created on demand when the user successfully completes the interactive authentication process. The `UserName` column identifies the authenticated user. The session state is set to `Active`, as this is the console session in which I ran the PowerShell command. The session name is a unique value indicating that this is a remote desktop connection.

Finally, session 2 lives on the physical console. It shows a state of `Connected`, as it currently hosts a `LogonUI` in case a user tries to physically log in to the machine. However, at this point there's no authenticated user in session 2, as you can see by the absence of a `UserName` in the listing.

Figure 12-4 summarizes the relationships between logon sessions and console sessions in this example. The console sessions are the gray boxes in the background, and the logon sessions are the white boxes in the foreground.

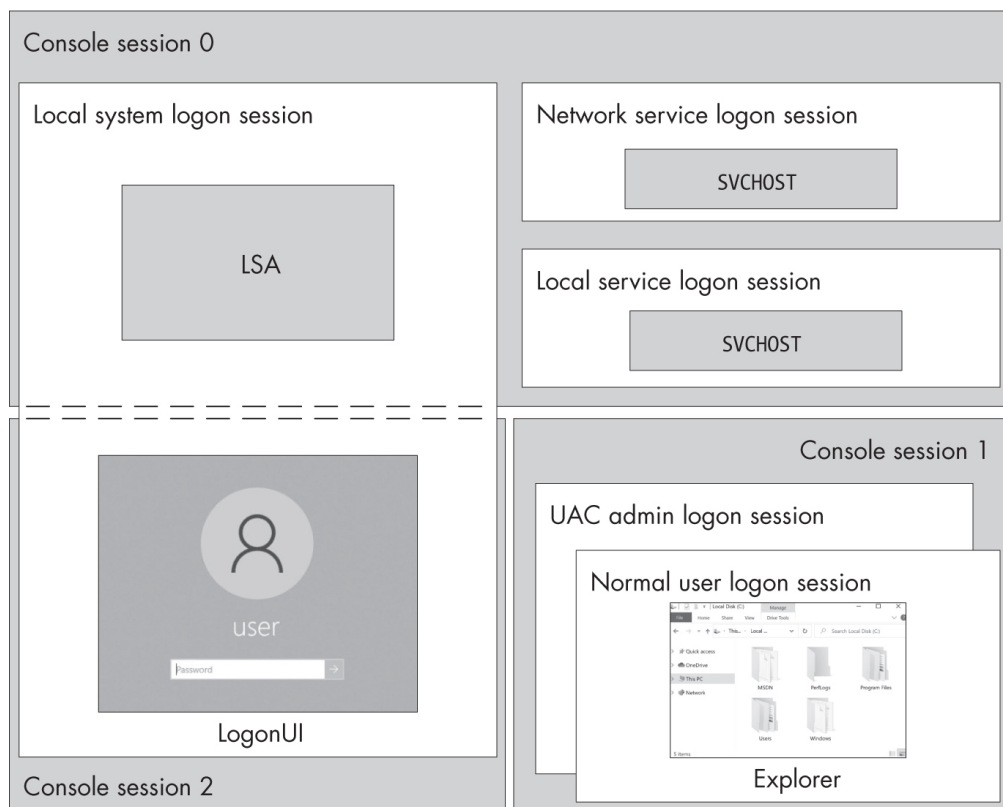


Figure 12-4: The console and logon sessions

Notice that console session 0 contains the service logon sessions, such as those for the local system, the network service, and the local service. The local system logon session is also used for the LogonUI process running in console session 2. At the bottom right is console session 1, which contains two user logon sessions: one for the UAC administrator and one for the filtered non-administrator.

NETWORK CREDENTIALS

One other important value stored in the logon session is the set of network authentication credentials for the user. Storing these credentials can save the user from having to retype them for every network service. Not all types of logon sessions store network credentials, though; for example, the `Interactive` and `Batch` logon types store the credentials, but the `Network` logon type does not. If you want a network logon session with stored network credentials, you can use the `NetworkCleartext` logon type instead.

The `NewCredentials` logon type doesn't authenticate a new user. Instead, the LSA makes a copy of the caller's token, creates a new logon session, and uses the supplied credentials only for network authentication. This allows a user to authenticate as a different user locally and remotely. Note that this logon type doesn't verify the credentials in the call to `LsaLogonUser`; it verifies them only when they're used. This means that if you specify the wrong credentials, `LsaLogonUser` will return successfully but then fail at a later point, when the credentials are required.

We'll cover network authentication and how it interacts with the user's network authentication credentials in more detail in the following chapters.

Token Creation

With a new logon session, the LSA can create the final `Token` object for the user. To do this, it must gather information about the token's various security properties, including the user's groups, privileges, and logon session ID, then pass these to `NtCreateToken`.

You might be wondering where the user's groups come from. As domain authentication is the most complex case, let's consider the groups as-

signed to a domain user token when Winlogon authenticates the user. (The group assignment will look similar in the local authentication process, except that the LSA will consider only local groups.) **Table 12-3** shows the group assignments for the *alice* user.

Table 12-3: Groups Added to an Interactive Token on a Domain-Joined System

Group name	Group source
<i>MINERAL\alice</i>	Domain user account
<i>MINERAL\Domain Users</i>	Domain group membership
<i>Authentication authority asserted identity</i>	
<i>NT AUTHORITY\Claims Valid</i>	
<i>MINERAL\Local Resource</i>	Domain-local resource group membership
<i>BUILTIN\Administrators</i>	Local group membership
<i>BUILTIN\Users</i>	
<i>NT AUTHORITY\INTERACTIVE</i>	Automatic LSA groups
<i>NT AUTHORITY\Authenticated Users</i>	
<i>Everyone</i>	
<i>Mandatory Label\High Mandatory Level</i>	
<i>NT AUTHORITY\LogonSessionId_0_6077548</i>	Winlogon groups
<i>LOCAL</i>	

As you can see, the groups added to the token come from six sources. The first entry comes from the domain user account. (In a local authentication scenario, the group would come from the local user account instead.)

Next are the domain group memberships. These come from the `Universal` and `Global` group scopes, discussed in the previous chapter. The *alice* user is a member of the first group, *Domain Users*. The other two groups are generated automatically when the user authenticates. The *Authentication authority asserted identity* group relates to a feature called *Service for User (S4U)*, which we'll explore when we talk about Kerberos authentication in [Chapter 14](#).

The following source includes the groups with the `DomainLocal` scope. These domain-local groups are marked in the token with the `Resource` group attribute, although the attribute doesn't affect their use in an access check. The list of domain-local resource groups a user belongs to is returned in the response from the `NetrLogonSamLogon` API, known as a *privilege attribute certificate (PAC)*. We'll also come back to the PAC in [Chapter 14](#).

Next, any local groups the user is a member of are added to the token. These local groups can be selected based on the domain SIDs provided during the authentication process.

These are followed by the automatic LSA groups. Membership in the *Everyone* and *Authenticated Users* groups is granted to all authenticated tokens automatically. *INTERACTIVE* group membership is granted when a user is authenticated using the `Interactive` logon type. [Table 12-4](#) provides a list of the SIDs added for different logon types. The LSA adds the *Mandatory Label\High Mandatory Level* SID automatically if the user is considered an administrator (for example, if they're in the *Administrators* group or have certain high-level privileges). This sets the integrity level of the token to `High`. Normal users get the *Medium Mandatory Level* SID, while system service users (such as *SYSTEM*) get the *System Mandatory Level* SID.

Table 12-4: The SIDs Added to the Token for Each Logon Type

Logon type	Name	SID
------------	------	-----

Logon type	Name	SID
Interactive	<i>NT AUTHORITY\INTERACTIVE</i>	S-1-5-4
Batch	<i>NT AUTHORITY\BATCH</i>	S-1-5-3
Service	<i>NT AUTHORITY\SERVICE</i>	S-1-5-6
Network	<i>NT AUTHORITY\NETWORK</i>	S-1-5-2
NetworkCleartext	<i>NT AUTHORITY\NETWORK</i>	S-1-5-2
NewCredentials	The same as that of the original token	N/A
RemoteInteractive	<i>NT AUTHORITY\INTERACTIVE</i>	S-1-5-4
	<i>NT AUTHORITY\REMOTE INTERACTIVE</i>	
	<i>LOGON</i>	S-1-5-14
Unlock	The same as the logon session that is being unlocked	N/A

Providing a unique SID for each logon type allows a security descriptor to secure resources depending on the type of logon. For example, a security descriptor could explicitly deny access to the *NT AUTHORITY\NETWORK* SID, meaning a user authenticated from the network would be denied access to the resource, while other authenticated users would be granted access.

The sixth set of SIDs added to the token are for the groups added by Winlogon when it calls the `LsaLogonUser` API. The API allows a caller with `SeTcbPrivilege` enabled to add arbitrary group SIDs to the created token, so Winlogon adds a logon session and a *LOCAL* SID. This logon session SID's

two RID values are the two 32-bit integers from a LUID generated by the `NtAllocateLocallyUniqueId` system call. You might assume that the LUID would match the one used for the logon session. However, as the SID is created before the call to the LSA that creates the logon session, this isn't possible. This SID is used to secure ephemeral resources such as the user's *BaseNamedObjects* directory.

NOTE

If you don't specify a logon session SID when creating the token, the LSA will add its own for you. However, it will follow the same pattern of using a different LUID from that of the token's logon session.

As discussed in [Chapter 10](#), the token's privileges are based on the account rights stored in the local LSA policy database. This is true even in domain authentication; however, the account rights can be modified using a domain group policy deployed to computers in the domain.

If the user is considered an administrator, UAC is enabled, and the user is authenticating with the `Interactive` OR `RemoteInteractive` logon type, the LSA will first build the full token and create a new logon session, then create a second copy of the full token with a new logon session but call `NtFilterToken` to remove administrator privileges (see [Chapter 4](#) for a more in-depth description of this). The LSA will then link the two tokens together and return the filtered token back to the caller. This behavior is why we observed two logon sessions for the same user in [Listing 12-2](#).

You can disable the token-splitting behavior by adjusting the system's UAC settings. It's also disabled by default for the *Administrator* user, which is always created when Windows is installed but only enabled by default on Windows Server systems. The LSA checks the last RID of the user's SID: if it's 500, which matches the *Administrator* user, the token won't be split.

Using the LsaLogonUser API from PowerShell

Now that you know how the `LsaLogonUser` API works, let's see how to access the API from the `NtObjectManager` PowerShell module. Unless you run PowerShell with `Set-LocalSecurity`, some features of the API will be blocked, such as adding new group SIDs to the token, but you'll be able to create a new token if you have the user's username and password.

We access the API via the `Get-NtToken` command and the `Logon` parameter.

Listing 12-4 shows how to use `Get-NtToken` to authenticate a new user.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
Password: *****
PS> $token = Get-NtToken -Logon -User user -Domain $env:COMPUTERNAME
-Password $password -LogonType Network
PS> Get-NtLogonSession -Token $token
```

LogonId	UserName	LogonType	SessionId
-----	-----	-----	-----
00000000-9BBFFF01	GRAPHITE\user	Network	3

Listing 12-4: Authenticating a user

It's best not to enter passwords on the command line. Instead, we use `Read-Host` with the `AsSecureString` property to read the password as a secure string.

We can then call `Get-NtToken`, specifying the username, the domain, and the password. (Replace the username in this example, `user`, with that of a valid local user.) We set the domain to the name of the local computer, indicating that we want to authenticate using a local account. You can set any logon type, but in this case we specify `Network`, which works for all users. Whether the LSA will allow other logon types depends on the assigned account rights.

NOTE

By default, the `LsaLogonUser` API won't authenticate a user with an empty password outside of the physical console. If you try running the command with a user account that has an empty password, the call will fail.

The logon type also determines what type of token `LsaLogonUser` will return based on the created token's likely purpose, such as creating a new process or impersonation. **Table 12-5** shows the mappings of logon type to token type. (We can freely convert between primary and impersonation tokens through duplication, so the tokens don't have to be used in the expected way.)

Table 12-5: Logon Types Mapped to Token Types

Logon type	Token type
Interactive	Primary
Batch	Primary
Service	Primary
Network	Impersonation
NetworkCleartext	Impersonation
NewCredentials	Primary
RemoteInteractive	Primary
Unlock	Primary

In [Listing 12-4](#), the command returned an impersonation token. You might be wondering: Are we allowed to impersonate the token without having `SeImpersonatePrivilege` enabled, especially if the token belongs to a different user? The LSA sets the new token's origin ID to the caller's authentication ID, so based on the rules for impersonation covered in [Chapter 4](#), we can, even if the token belongs to a different user.

This isn't considered a security issue, because if you know the user's password, you can already fully authenticate as that user. In [Listing 12-5](#), we check whether the origin and authentication IDs match using the `Get-NtTokenId` command.

```
PS> Get-NtTokenId -Authentication
LUID
----
00000000-000A0908

PS> Get-NtTokenId -Token $token -Origin
LUID
```

```
-----  
00000000-000A0908
```

Listing 12-5: Comparing the authentication ID and origin ID

We query the primary token for its authentication ID, then query the new token for its origin ID. The output shows that the IDs are equal.

However, there is one restriction on impersonating the token. If the user being authenticated is an administrator, and the authentication process uses a logon type other than `Interactive`, the command won't return a filtered token. Instead, it will return an administrator with a `High` integrity level. This integrity level prevents the token from being impersonated from a `Medium`-level process. But because the returned token handle has write access, we can reduce the integrity level to `Medium` before impersonating it. We do this in [Listing 12-6](#).

```
PS> Get-NtTokenIntegrityLevel -Token $token  
High  
  
PS> Test-NtTokenImpersonation $token  
False  
  
PS> Set-NtTokenIntegrityLevel -Token $token Medium  
PS> Test-NtTokenImpersonation $token  
True
```

Listing 12-6: Testing the ability to impersonate the returned token

In this case, the token we've authenticated is a member of the *Administrators* group and so has a `High` integrity level. We try to impersonate it, and as you can see, the command returns `False`. We then set the token's integrity level to `Medium` and test impersonation again. The operation now returns `True`.

Creating a New Process with a Token

If you use a logon type that returns a `Primary` token, you might assume that the token will enable you to create a new process. To test this, run [Listing](#)

[12-7](#) as a non-administrator user, making sure to change the username to that of a valid account.

```
PS> $token = Get-NtToken -Logon -User user -Domain $env:COMPUTERNAME  
-Password $password -LogonType Interactive  
PS> New-Win32Process cmd.exe -Token $token  
Exception calling "CreateProcess": "A required privilege is not held  
by the client"
```

Listing 12-7: Creating a new process with an authenticated token

You'll find that creating the new process fails. This is because the new token doesn't meet the requirements for primary token assignment described in [Chapter 4](#). The process creation would work if the calling process had `SeAssignPrimaryTokenPrivilege`, which Winlogon would have; however, a normal user process doesn't have this privilege.

If you rerun the command as an administrator, though, it should succeed, even though administrators are not granted the privilege by default. Let's explore why this works. The `New-Win32Process` command first tries to create the process using the `CreateProcessAsUser` API, which runs in-process. As the calling process doesn't have `SeAssignPrimaryTokenPrivilege`, this operation fails.

Upon this failure, the `New-Win32Process` API will fall back to calling an alternative API, `CreateProcessWithToken`. This API isn't implemented in-process; instead, it's implemented in a system service, the secondary logon service, which does have `SeAssignPrimaryTokenPrivilege`. In this case, the service will check whether the caller has `SeImpersonatePrivilege` before creating the new process.

The command therefore works for administrators who are granted `SeImpersonatePrivilege`. Even so, administrators shouldn't rely on `CreateProcessWithToken` exclusively, because the API doesn't support many features of `CreateProcessAsUser`, such as inheriting arbitrary handles to the new process.

There is also a way for a non-administrator user to create a process as a different user. The secondary logon service exposes a second API, `CreateProcessWithLogon`, that accepts the username, domain, and password

for the user to create instead of a token handle. The service authenticates the user using `LsaLogonUser`, then uses the authenticated token with `CreateProcessAsUser`. As the service has `SeAssignPrimaryTokenPrivilege`, the process creation will succeed.

You can specify the `Credential` parameter when calling the `New-Win32Process` command to use `CreateProcessWithLogon`, as shown in [Listing 12-8](#).

```
PS> $creds = Read-LsaCredential
UserName: alice
Domain: MINERAL
Password: *****

PS> $proc = New-Win32Process -CommandLine cmd.exe -Credential $creds
PS> $proc.Process.User
Name          Sid
----          -
MINERAL\alice S-1-5-21-1195776225-522706947-2538775957-1110
```

Listing 12-8: Calling `CreateProcessWithLogon` using `New-Win32Process`

Here we read the credentials for the *alice* user and create the new process using `New-Win32Process`, specifying the credentials with the `Credential` parameter. This will call the `CreateProcessWithLogon` API.

The API will return a process and thread handle to use. For example, we can query for the process user, which shows it was created with a token for the authenticated *alice* user.

The API doesn't allow you to specify the logon type of the user (it defaults to `Interactive`), but you can specify the `NetCredentialsOnly` flag to the `LogonFlags` parameter to use the `NewCredentials` logon type instead.

The Service Logon Type

Let's wrap up this chapter by talking a little more about the `Service` logon type. The service control manager uses this logon type to create tokens for system service processes. It will allow any user account that has been granted the `SeServiceLogonRight` account right to authenticate.

However, the LSA also supports four well-known local service accounts that are not stored in the SAM database. We can create them using `LsaLogonUser` by specifying the domain name as `NT AUTHORITY` with the `Service` logon type and one of the usernames in [Table 12-6](#), which also shows the user SIDs.

Table 12-6: Usernames and SIDs for the `Service` Logon Type

Username	User SID
<i>IUSR</i>	S-1-5-17
<i>SYSTEM</i>	S-1-5-18
<i>LOCAL SERVICE</i> or <i>LocalService</i>	S-1-5-19
<i>NETWORK SERVICE</i> or <i>NetworkService</i>	S-1-5-20

The *SYSTEM* user is the only administrator of the four users; the other three are not members of the *Administrators* group, but they do have high-level privileges such as `SeImpersonatePrivilege`, which makes them effectively equivalent to an administrator.

The *IUSR* account represents the anonymous internet user. It's available to reduce the privileges for the Internet Information Services (IIS) web server when it's configured for anonymous authentication. When a request is made to the IIS web server with no user credentials, it will impersonate an *IUSR* account token before opening any resources, such as files. This prevents inadvertently exposing resources remotely as a privileged user.

For these built-in service accounts, you don't need to specify a password, but you do need to call `LsaLogonUser` with `SeTcbPrivilege` enabled, which prevents it from being used outside of a system service. [Listing 12-9](#) shows how to use `Get-NtToken` to create a *SYSTEM* user token. Run these commands as an administrator.

```
PS> Get-NtToken -Logon -LogonType Service -Domain 'NT AUTHORITY' -User SYSTEM  
-WithTcb
```

User	GroupCount	PrivilegeCount	AppContainer	Restricted
NT AUTHORITY\SYSTEM	11	31	False	False


```
PS> Get-NtToken -Service System -WithTcb
```

User	GroupCount	PrivilegeCount	AppContainer	Restricted
NT AUTHORITY\SYSTEM	11	31	False	False

Listing 12-9: Getting the SYSTEM user token

Even as an administrator you don't receive `SeTcbPrivilege` by default, so the command supports a `WithTcb` parameter, which automatically impersonates a token with the privilege enabled. You can also simplify the creation of a service account by using the `Service` parameter and specifying the name of the service user to create.

Worked Examples

Let's walk through some examples that demonstrate how to use the various commands introduced in this chapter in security research or systems analysis.

Testing Privileges and Logon Account Rights

I mentioned in [Chapter 10](#) that you can use the `Add-NtAccountRight` command to add a SID to the list of account rights. Now that we know how to authenticate a user, let's use this command to explore these account rights. In [Listing 12-10](#), we assign privileges and logon account rights to a new user. Run these commands as an administrator.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
Password: *****
❶ PS> $user = New-LocalUser -Name "Test" -Password $password
PS> $sid = $user.Sid.Value
❷ PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Interactive
PS> $token.ElevationType
Default

PS> $token.Close()
```



```

❸ PS> Add-NtAccountRight -Privilege SeDebugPrivilege -Sid $sid
PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
        -LogonType Interactive
PS> Enable-NtTokenPrivilege -Token $token SeDebugPrivilege -PassThru
❹ WARNING: Couldn't set privilege SeDebugPrivilege

PS> $token.ElevationType
Limited

PS> $token.Close()
❺ PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
        -LogonType Network
PS> Enable-NtTokenPrivilege -Token $token SeDebugPrivilege -PassThru
Name                Luid                Enabled
----                -
SeDebugPrivilege 00000000-00000014 True

PS> $token.ElevationType
Default

PS> $token.Close()
❻ PS> Add-NtAccountRight -LogonType SeDenyInteractiveLogonRight -Sid $sid
PS> Add-NtAccountRight -LogonType SeBatchLogonRight -Sid $sid
❼ PS> Get-NtToken -Logon -User $user.Name -SecurePassword $password
        -LogonType Interactive
Get-NtToken : (0x80070569) - Logon failure: the user has not been granted
the requested logon type at this computer.

PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
        -LogonType Batch
PS> Get-NtTokenGroup $token | Where-Object {$_.Sid.Name -eq
"NT AUTHORITY\BATCH"}
Sid                : S-1-5-3
Attributes         : Mandatory, EnabledByDefault, Enabled
Enabled            : True
Mandatory          : True
DenyOnly           : False
❽ Name              : NT AUTHORITY\BATCH

PS> $token.Close()
❿ PS> Remove-NtAccountRight -Privilege SeDebugPrivilege -Sid $sid
PS> Remove-NtAccountRight -LogonType SeDenyInteractiveLogonRight -Sid $sid
PS> Remove-NtAccountRight -LogonType SeBatchLogonRight -Sid $sid
PS> Remove-LocalUser $user

```

Listing 12-10: Assigning account rights to a new user

We start by creating a new user ❶ and testing that we can authenticate interactively ❷. We can do so because the user is automatically part of the *BUILTIN\Users* group, which has *SeInteractiveLogonRight* by default. We also check that the token hasn't been filtered for UAC by looking at the *ElevationType* parameter, which shows up as *Default*, indicating that no filtering took place.

Next, we assign the user the *SeDebugPrivilege* privilege ❸. This is a high-level privilege, so we should expect the LSA to perform UAC filtering. We find this to be the case when we authenticate the user: we can't enable *SeDebugPrivilege*, since it's been filtered ❹, and the *ElevationType* is now set to *Limited*.

However, we can instead use network authentication ❺, which isn't subject to the default UAC filtering rules. We can now enable *SeDebugPrivilege* ❻, and the *ElevationType* becomes *Default* once again, indicating that no filtering took place.

We then test the logon account rights. Remember that the user is granted *SeInteractiveLogonRight* because they are a member of the *BUILTIN\Users* group. We can't remove that logon right without also removing them from that group, so instead we explicitly deny it to the specific user by adding their SID to the *SeDenyInteractiveLogonRight* ❼. Then we verify the intended behavior by trying to log on interactively ❽, which now returns an error.

We also added the user's SID to the *SeBatchLogonRight*, which allows them to authenticate as a batch logon session. Normally, only members of the *Administrators* group receive this access right. We verify we've authenticated as a batch logon session by checking for the *NT AUTHORITY\BATCHE* group that the LSA assigns ❾.

Finally, we clean up the account right assignments using the *Remove-NtAccountRight* command ❿. This isn't strictly necessary, as the LSA will clean up the assignments when the local user is removed, but I've included the operations here to demonstrate the use of the command.

Creating a Process in a Different Console Session

In certain scenarios, you might want to start a process inside a different console session. For example, if you're running code in a system service using session 0, you might want to show a message on the currently authenticated user's desktop.

To successfully create a process on another desktop, you need `SeTcbPrivilege` to change a token's session ID and `SeAssignPrimaryTokenPrivilege` to create the process. By default, an administrator user has neither of these privileges, so to test the example code provided here you'll need to run PowerShell as the *SYSTEM* user.

First run the following command as an administrator to create a shell process on your desktop with the required privileges:

```
PS> Start-Win32ChildProcess ((Get-NtProcess -Current).Win32ImagePath)
-RequiredPrivilege SeTcbPrivilege,SeAssignPrimaryTokenPrivilege
```

Next, make sure that you have two users authenticated at the same time on different desktops on the same machine. If you use Fast User Switching, you'll be able to easily confirm that a process was created on each desktop.

Listing 12-11 starts by finding the console session for the new process. Run these commands as the *SYSTEM* user.

```
PS> $username = "GRAPHITE\user"
❶ PS> $console = Get-NtConsoleSession |
Where-Object FullyQualifiedUserName -eq $username
❷ PS> $token = Get-NtToken -Duplicate -TokenType Primary
PS> Enable-NtTokenPrivilege SeTcbPrivilege
PS> $token.SessionId = $console.SessionId
PS> $cmd = "cmd.exe"
❸ PS> $proc = New-Win32Process $cmd -Token $token -Desktop "WinSta0\Default"
-CreationFlags NewConsole
❹ PS> $proc.Process.SessionId -eq $console.SessionId
True
```

```
PS> $proc.Dispose()
PS> $token.Close()
```

Listing 12-11: Creating a new process in a different console session

We start by selecting the console session belonging to a user named *GRAPHITE\user* ❶. We then create a duplicate of our current token (which belongs to the *SYSTEM* user), enable *SeTcbPrivilege*, and assign the console session ID to the token ❷.

With this new token, we can create a new process using the *New-Win32Process* command, specifying the *Token* parameter ❸. In this case we're creating a copy of Notepad, but you can change this process to any application you'd like by altering the command. Also note that we set the name of the window station and desktop, separated by a backslash, for the new process. Using *WinSta0* and *Default*, respectively, ensures that we create the application on the default desktop; otherwise, the user interface would be hidden.

We can verify that we've created the process in the target session by comparing the expected session ID with the actual session ID assigned to the process ❹. In this case, the comparison returns *True*, which indicates success. If you now switch back to the other user, you should find a copy of Notepad running as the *SYSTEM* user on the desktop.

Authenticating Virtual Accounts

In [Chapter 10](#), I mentioned that you can create your own SID-to-name mappings in the LSA using the *Add-NtSidName* command. Once you've set up a mapping, you can also create a new token for that SID through *LsaLogonUser*. [Listing 12-12](#) demonstrates; run these commands as an administrator.

```
PS> $domain_sid = Get-NtSid "S-1-5-99" ❶
PS> $group_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 0
PS> $user_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 1
PS> $domain = "CUSTOMDOMAIN"
PS> $group = "ALL USERS"
PS> $user = "USER"
PS> $token = Invoke-NtToken -System {❷
```

```

Add-NtSidName -Domain $domain -Sid $domain_sid -Register ❶
Add-NtSidName -Domain $domain -Name $group -Sid $group_sid -Register
Add-NtSidName -Domain $domain -Name $user -Sid $user_sid -Register
Add-NtAccountRight -Sid $user_sid -LogonType SeInteractiveLogonRight ❷
Get-NtToken -Logon -Domain $domain -User $user -LogonProvider Virtual
-LogonType Interactive ❸
Remove-NtAccountRight -Sid $user_sid -LogonType SeInteractiveLogonRight ❹
Remove-NtSidName -Sid $domain_sid -Unregister
}
PS> Format-NtToken $token -User -Group
USER INFORMATION
-----
Name                Sid
----                -
CUSTOMDOMAIN\User  S-1-5-99-1 ❺

GROUP SID INFORMATION
-----
Name                Attributes
----                -
Mandatory Label\Medium Mandatory Level Integrity, IntegrityEnabled
Everyone            Mandatory, EnabledByDefault, Enabled
BUILTIN\Users       Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\INTERACTIVE Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\Authenticated Users Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\This Organization Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\LogonSessionId_0_10173 Mandatory, EnabledByDefault, Enabled, Logc
CUSTOMDOMAIN\ALL USERS Mandatory, EnabledByDefault, Enabled ❻

```

Listing 12-12: Creating a virtual account token

We start by setting up some parameters to use in later commands ❶. We create three SIDs: the domain, a group, and a user. These values don't need to reflect real SIDs or names. We then need to add the SIDs and create a token, all of which requires `SeTcbPrivilege`, so we impersonate a `SYSTEM` token ❷.

We register the three SIDs using the `Add-NtSidName` command ❸. Note that you must specify the `Register` parameter; otherwise, you'll merely add the SID to the PowerShell module's name cache and won't register it with LSASS. Once we've added the SIDs, we need to grant the user `SeInteractiveLogonRight` so that we can authenticate them and receive a to-

ken ④. You could choose a different logon right, such as `SeServiceLogonRight`, if you wanted.

We can now authenticate the user via `LsaLogonUser` by using `Get-NtToken` ⑤. Make sure to specify the `Virtual` logon provider and the `Interactive` logon type. You don't need to specify a password, but you can't perform the operation without `SeTcbPrivilege`.

Before we finish impersonating, we remove the logon right and then delete the domain SID ⑥. Deleting the domain SID will also delete the group and user SIDs automatically.

Finally, we format the token. Now we can see that the user SID is the virtual SID we created ⑦, and that the token is automatically granted the group SID as well ⑧. Note that if we hadn't added the SID-to-name mapping for the group SID, we'd still be granted it, but the SID would not be resolvable to a name. We can now impersonate the token or use it to create a new process running under that user identity.

Wrapping Up

As you've seen, interactive authentication, the process used to access the Windows desktop, is an extremely complicated topic. The authentication process requires a combination of a user interface, which collects the credentials, and the Winlogon process, which calls the LSA's `LsaLogonUser` API. Once the API has validated the user's credentials, it creates a new logon session, along with a token that Winlogon can use to create the user's initial processes. The logon session can also cache the credentials so the user won't need to re-enter them to access network services.

Next, we defined the differences between local authentication and domain authentication. We only touched on how authentication works with Netlogon here, but we'll cover the more common Kerberos in [Chapter 14](#). With an understanding of the basic authentication mechanisms in hand, we discussed how the LSA uses the user information to build a token, including how it assigns groups and privileges and how UAC results in token filtering for administrators.

We then discussed how to call the `LsaLogonUser` API using the PowerShell module's `Get-NtToken` command. We saw that we can use the token returned from the API to impersonate a user, because the LSA sets the token's origin ID to the caller's authentication ID. We also saw how to create a new process as a different user via the `CreateProcessWithLogon` API, exposed through the `New-Win32Process` command.

Finally, we looked briefly at the `Service` logon type and the four accounts that the LSA predefines. The service control manager uses these for its system service processes. In the next chapter, we'll begin exploring how network authentication allows a user to authenticate to another Windows system. This will also allow us to understand the protocols used by domain authentication.